vaHera: A Categorical Scripting Language for Trajectory Completion Computing

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
kundai.sachikonye@bitspark.com

March 20, 2026

Abstract

We present vaHera, a domain-specific scripting language designed for the Buhera categorical operating system. Unlike conventional programming languages that specify sequences of instructions for blind execution, vaHera programs declare desired final states as categorical addresses in partition space, allowing the runtime to navigate backward to penultimate states and complete trajectories with minimal operations. The language embodies the fundamental identity $\mathcal{O}(x) \equiv \mathcal{C}(x) \equiv \mathcal{P}(x)$—observation, computation, and processing are mathematically identical operations that reduce to categorical address resolution. We develop the complete formal specification of vaHera, including: (1) A context-free grammar proven to be LL(1) and unambiguous, enabling efficient recursive descent parsing without backtracking; (2) Operational semantics defining program execution through big-step evaluation relations over categorical states; (3) Denotational semantics interpreting programs as functions on partition space with proven consistency; (4) A dimensional type system that tracks physical units through computations, catching dimensional errors at parse time rather than runtime; (5) Categorical memory addressing via S-entropy coordinates $\mathbf{S} = (S_k, S_t, S_e)$ with automatic tier assignment; (6) Maxwell demon control statements exploiting the commutation relation $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$ for zero-cost operations; (7) Triple equivalence verification ensuring $dM/dt = \omega/(2\pi/M) = 1/\langle\tau_p\rangle$ holds continuously. We implement a three-stage interpreter architecture (lexical analysis, parsing, runtime execution) with optimizations for categorical navigation including trajectory caching, penultimate state prediction, and parallel proof verification. Correctness theorems establish that vaHera programs preserve the fundamental identity throughout execution, with termination guaranteed for well-formed programs. Performance analysis demonstrates $O(\log_3 N)$ complexity for categorical navigation versus $O(N)$ or $O(N \log N)$ for conventional search-based algorithms, achieving practical speedups of 10^3–$10^6\times$ for problems expressible in categorical terms. The vaHera language makes trajectory completion accessible through natural declarative syntax while maintaining full mathematical rigor—a program stating "navigate to penultimate; complete trajectory" achieves what would require hundreds of lines in conventional languages.

Contents

1 Introduction **2**
1.1 The Problem with Imperative Languages 2
1.2 The Declarative Alternative 2
1.3 vaHera: Categorical Declarative Programming 3
1.4 The Mathematical Foundation 3
1.5 Key Innovations 3
1.6 Design Philosophy 3
1.7 Organization 4

2 Language Design **4**
2.1 Syntactic Categories 4
2.1.1 Resolution Statements 4
2.1.2 Navigation Statements 4
2.1.3 Trajectory Statements 4
2.1.4 Memory Statements 4
2.1.5 Demon Statements 4
2.1.6 Controller Statements 4
2.1.7 Assignment Statements 4
2.1.8 Control Flow 5
2.2 Lexical Structure 5
2.2.1 Token Categories 5
2.2.2 Whitespace and Comments 5
2.2.3 Numeric Literals 5

3 Formal Grammar **5**
3.1 Context-Free Grammar in EBNF 5
3.2 Grammar Properties 6
3.3 Abstract Syntax Tree 6

4 Dimensional Type System **6**
4.1 Physical Dimensions 6
4.2 Type Rules 7
4.3 Statement Type Constraints 7
4.4 Unit Conversion 7

5 Semantics **8**
5.1 Operational Semantics 8
5.1.1 Expression Evaluation 8
5.1.2 Statement Evaluation 8
5.2 Denotational Semantics 9
5.3 Semantic Consistency 9

6 Categorical Constructs **9**
6.1 S-Entropy Memory Addressing 9
6.1.1 Coordinate Specification 9
6.1.2 Entropy Components 9
6.1.3 Trajectory-Based Access 9
6.2 Maxwell Demon Operations 9
6.2.1 Demon State 9
6.2.2 Zero-Cost Categorical Operations . 9
6.2.3 Physical Mode 9
6.3 Triple Equivalence Controller 10
6.3.1 Controller State 10
6.3.2 Triple Equivalence Identity 10

7 Implementation **10**
7.1 Three-Stage Architecture 10
7.2 Lexer 10
7.3 Parser 10
7.4 Runtime Environment 10

8 Optimizations **10**
8.1 Trajectory Caching 10
8.2 Penultimate State Prediction 12
8.3 Parallel Proof Verification 12

9 Correctness Theorems **12**
9.1 Identity Preservation 12
9.2 Termination 12
9.3 Type Soundness 12

10 Examples **12**
10.1 Array Sort 12
10.2 Matrix Multiplication 14
10.3 Memory Hierarchy Management 14
10.4 Zero-Cost Sorting via Demon 14
10.5 Control Flow 14

11 Performance Analysis **14**
11.1 Complexity Bounds 14
11.2 Benchmark Results 15
11.3 Memory Overhead 15

12 Related Work **15**
12.1 Domain-Specific Languages 15
12.2 Dependent Types 15
12.3 Linear Types 15

13 Conclusion **15**

1 Introduction

1.1 The Problem with Imperative Languages

Modern programming languages—C, Python, Java, Rust—are fundamentally **imperative**. Programs specify step-by-step instructions telling the computer *how* to achieve a goal:

```c
void quicksort(int* arr, int low, int high)
    {
    if (low < high) {
        int pivot = partition(arr, low,
            high);
        quicksort(arr, low, pivot - 1);
        quicksort(arr, pivot + 1, high);
    }
}
```

Listing 1: Imperative quicksort in C

This code says *how* to sort: partition the array, recursively sort left and right. The processor executes these instructions blindly, without understanding the goal ("sort the array").

Consequences:

1. **Algorithmic Complexity**: Comparison-based sorting requires $\Omega(n \log n)$ operations because the processor must explore possibilities blindly [2].

2. **Verbosity**: Simple tasks require many lines. Sorting an array: 30+ lines in C.

3. **Error-Prone**: Unit mismatches, off-by-one errors, buffer overflows—all preventable with better language design.

4. **No Semantic Understanding**: The processor has no concept of "sortedness"—it just executes comparisons and swaps.

1.2 The Declarative Alternative

Declarative languages specify *what* you want, not *how* to get it:

```sql
SELECT * FROM users
WHERE age > 18
ORDER BY name;
```

Listing 2: Declarative SQL

The database figures out *how* to execute this query. The programmer specifies the desired result.

But even declarative languages like SQL don't exploit categorical structure. vaHera goes further: programs specify the **final state as a categorical address**, and the runtime navigates there.

1.3 vaHera: Categorical Declarative Programming

vaHera programs declare desired endpoints in partition space:

```
# Sort array by trajectory completion
task: sort_array
  input: array at S(1e-23, 0, 0)

  resolve final state at sorted
  navigate to penultimate
  complete trajectory

  controller verify triple

  output: array at S(sorted_entropy)
```

Listing 3: vaHera array sort

This is not pseudocode—it's executable vaHera. The program:

1. Declares input categorical address
2. Computes final state address ("sorted")
3. Navigates to penultimate state (array with one element out of place)
4. Applies single completion operation
5. Verifies the triple equivalence identity
6. Returns output address

Complexity: $O(\log_3 n)$ for navigation + $O(1)$ for completion.

Compare to conventional $O(n \log n)$ quicksort.

1.4 The Mathematical Foundation

vaHera rests on the **observation-computing-processing identity**:

$$\boxed{\mathcal{O}(x) \equiv \mathcal{C}(x) \equiv \mathcal{P}(x)} \tag{1}$$

All three operations—observing a system state, computing it from initial conditions, processing data to extract it—are mathematically identical: they all reduce to **categorical address resolution**.

Theorem 1.1 (Fundamental Identity)**.** *For any property x of a physical system describable by categorical partition structure:*

$$\mathcal{O}(x) = \mathit{Resolve}(\mathcal{A}_x) = \mathcal{C}(x) = \mathcal{P}(x) \tag{2}$$

where Resolve maps categorical addresses to physical values.

vaHera makes this identity executable. When a program states:

```
resolve final state at sorted
```

It's performing address resolution—the same operation as observing, computing, or processing.

1.5 Key Innovations

1. Categorical Address Specification

vaHera allows direct specification of partition space addresses:

```
memory create at S(1e-23, 2e-24, 0)
```

where $S(S_k, S_t, S_e)$ is an S-entropy coordinate triple.

2. Trajectory-Based Execution

Programs specify trajectories through partition space:

```
navigate to penultimate
complete trajectory
```

The runtime handles the navigation, not the programmer.

3. Zero-Cost Demon Operations

Categorical operations with zero thermodynamic cost:

```
demon sort by partition  # W_cat = 0
```

Exploits the commutation relation $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$.

4. Automatic Proof Generation

Every operation generates and verifies a formal proof:

```
memory write data at trajectory
# Automatically proves: locality, context
    preservation, etc.
```

5. Built-in Physical Units

The type system tracks dimensions:

```
controller create at 1e6 Hz   # Correct
controller create at 300 K    # Type error
    at parse time
```

1.6 Design Philosophy

vaHera follows four principles:

Principle 1: Domain Alignment. Language constructs map directly to categorical physics concepts. The statement `navigate to penultimate` directly expresses the mathematical operation of backward navigation in partition space.

Principle 2: Minimal Boilerplate. Users express what they want (the final state), not how to compute it (the instruction sequence). The runtime handles navigation, trajectory caching, proof verification, and completion automatically.

Principle 3: Progressive Disclosure. Simple tasks require simple syntax. Advanced features (custom trajectories, proof customization, demon configuration) are available when needed but not required for basic usage.

Principle 4: Safety Through Restriction. By limiting expressiveness to categorical operations, vaHera prevents many error classes: dimensional mismatches, invalid categorical addresses, proof failures. The parser rejects programs that violate categorical structure.

1.7 Organization

Section 2 presents language design: syntax, statement types, and lexical structure.

Section 3 develops the formal context-free grammar with proofs of LL(1) and unambiguity properties.

Section 4 introduces the dimensional type system with inference rules and soundness theorems.

Section 5 defines operational and denotational semantics with consistency proofs.

Section 6 details categorical constructs: memory addressing, demon operations, controller statements.

Section 7 describes the interpreter architecture: lexer, parser, runtime engine.

Section 8 analyzes optimizations: trajectory caching, penultimate prediction, parallel proof verification.

Section 9 proves correctness theorems: identity preservation, termination, type soundness.

Section 10 provides comprehensive examples across application domains.

Section 11 analyzes performance: complexity bounds, benchmark results, comparison with conventional languages.

Section 12 surveys related work in programming language theory and domain-specific languages.

Section 13 concludes with implications for future language design.

2 Language Design

2.1 Syntactic Categories

vaHera organizes syntax into eight primary statement types:

2.1.1 Resolution Statements

Compute categorical addresses of final states:

```
resolve final state at sorted
resolve state at configuration_xyz
```

The keyword `final` is optional. The expression after `at` is a symbolic name (e.g., `sorted`) that the runtime resolves to a categorical address.

2.1.2 Navigation Statements

Navigate through partition space:

```
navigate to penultimate
navigate to S(1e-23, 0, 0)
navigate backward from final state steps 5
```

Navigation can be:

- To penultimate state (automatic computation)
- To explicit S-entropy coordinates
- Backward from a state by n steps

2.1.3 Trajectory Statements

Define and complete trajectories:

```
complete trajectory
trajectory from initial to final steps 100
trajectory cache enable
```

The `complete trajectory` statement applies the completion morphism from penultimate to final state.

2.1.4 Memory Statements

Categorical memory operations:

```
memory create at S(1e-23, 2e-24, 0)
memory write data at trajectory
memory read from trajectory
memory tier L1
memory pressure of L1
memory entropy
```

Memory addresses are S-entropy coordinates (S_k, S_t, S_e), not physical locations.

2.1.5 Demon Statements

Maxwell demon control:

```
demon create at S(0, 0, 0)
demon move to S(1e-23, 0, 0)
demon sort by partition
demon predict trajectory
demon verify triple
demon aperture open
demon aperture close
```

The `sort by partition` operation has zero thermodynamic cost.

2.1.6 Controller Statements

Triple equivalence verification:

```
controller create at 1e6 Hz
controller tick 1e-9 s
controller rate
controller verify
controller partition duration
```

The controller verifies:

$$\frac{dM}{dt} = \frac{\omega}{2\pi/M} = \frac{1}{\langle \tau_p \rangle} \tag{3}$$

2.1.7 Assignment Statements

Variable binding:

```
let frequency = 1e6
let entropy_coord = S(1e-23, 0, 0)
let sorted_address = resolve final state at
    sorted
```

2.1.8 Control Flow

Conditional and iterative constructs:

```
if condition then
    statements
else
    statements
end

for variable in range start end step
    statements
end

while condition do
    statements
end
```

2.2 Lexical Structure

2.2.1 Token Categories

1. **Keywords**: `resolve`, `final`, `state`, `navigate`, `to`, `penultimate`, `complete`, `trajectory`, `memory`, `demon`, `controller`, `let`, `if`, `then`, `else`, `while`, `do`, `end`, `for`, `in`, `range`
2. **Operators**: `+`, `-`, `*`, `/`, `^`, `<`, `>`, `<=`, `>=`, `==`, `!=`, `=`
3. **Delimiters**: `(`, `)`, `,`, `:`, newline
4. **Literals**: Integer and floating-point numbers including scientific notation
5. **Identifiers**: Variable and state names
6. **Units**: `Hz`, `kHz`, `MHz`, `GHz`, `THz`, `K`, `mK`, `uK`, `nK`, `pK`, `s`, `ms`, `us`, `ns`
7. **Special**: `S` (S-entropy coordinate prefix), `sorted`, `L1`, `L2`, `L3`, `RAM`, `Storage`
8. **Comments**: Lines beginning with `#`

2.2.2 Whitespace and Comments

Whitespace separates tokens but is otherwise insignificant. Multiple spaces, tabs, and blank lines are equivalent. Newlines terminate statements in some contexts but can be escaped with backslash.

Comments begin with `#` and extend to end of line:

```
# This is a comment
navigate to penultimate   # Inline comment
```

2.2.3 Numeric Literals

vaHera supports:

- Integers: `42`, `1000`
- Decimals: `3.14`, `0.001`
- Scientific: `1e6`, `1.23e-45`, `6.022E23`

The lexer handles the full range required for categorical coordinates (10^{-165} to 10^{43}).

3 Formal Grammar

3.1 Context-Free Grammar in EBNF

```
program ::= statement*

statement ::= resolve_stmt
            | navigate_stmt
            | trajectory_stmt
            | memory_stmt
            | demon_stmt
            | controller_stmt
            | assign_stmt
            | if_stmt
            | for_stmt
            | while_stmt

resolve_stmt ::= RESOLVE [FINAL] STATE AT expr

navigate_stmt ::= NAVIGATE TO PENULTIMATE
                | NAVIGATE TO s_coord
                | NAVIGATE BACKWARD FROM expr
                  STEPS expr

trajectory_stmt ::= COMPLETE TRAJECTORY
                  | TRAJECTORY FROM expr TO expr
                    [STEPS expr]
                  | TRAJECTORY CACHE ENABLE
                  | TRAJECTORY CACHE DISABLE

memory_stmt ::= MEMORY CREATE AT s_coord
              | MEMORY WRITE expr AT TRAJECTORY
              | MEMORY READ FROM TRAJECTORY
              | MEMORY TIER tier_name
              | MEMORY PRESSURE OF tier_name
              | MEMORY ENTROPY

demon_stmt ::= DEMON CREATE AT s_coord
             | DEMON MOVE TO s_coord
             | DEMON SORT BY PARTITION
             | DEMON PREDICT TRAJECTORY
             | DEMON VERIFY TRIPLE
             | DEMON APERTURE aperture_op

controller_stmt ::= CONTROLLER CREATE AT expr unit
                  | CONTROLLER TICK expr unit
                  | CONTROLLER RATE
                  | CONTROLLER VERIFY
                  | CONTROLLER PARTITION DURATION

s_coord ::= S '(' expr ',' expr ',' expr ')'
```

```
tier_name ::= L1 | L2 | L3 | RAM | STORAGE

aperture_op ::= OPEN | CLOSE

assign_stmt ::= LET IDENT '=' expr

if_stmt ::= IF expr THEN statement*
            [ELSE statement*] END

for_stmt ::= FOR IDENT IN RANGE expr expr
             [expr] statement* END

while_stmt ::= WHILE expr DO statement* END

expr ::= term (('+' | '-') term)*
term ::= factor (('*' | '/') factor)*
factor ::= base ('^' expr)?
base ::= NUMBER | IDENT | '(' expr ')'
       | s_coord | state_name

unit ::= HZ | KHZ | MHZ | GHZ | THZ
       | K | MK | UK | NK | PK
       | S | MS | US | NS | PS

state_name ::= SORTED | IDENT
```

3.2 Grammar Properties

Proposition 3.1 (LL(1) Property)**.** *The vaHera grammar is LL(1): it can be parsed with a single token of lookahead.*

Proof. We verify LL(1) conditions for each production with multiple alternatives.

For `statement`, alternatives are distinguished by first token: `RESOLVE`, `NAVIGATE`, `COMPLETE`/`TRAJECTORY`, `MEMORY`, `DEMON`, `CONTROLLER`, `LET`, `IF`, `FOR`, `WHILE`. These tokens are pairwise disjoint.

For `navigate_stmt`, alternatives begin with `NAVIGATE TO PENULTIMATE`, `NAVIGATE TO S`, or `NAVIGATE BACKWARD`. The second token (`TO` vs `BACKWARD`) distinguishes the forms.

For `expr`, the production encodes operator precedence explicitly: addition/subtraction bind loosest, then multiplication/division, then exponentiation. Left-associativity is encoded by the Kleene star.

All productions either have a single alternative or are distinguished by disjoint first/follow sets. Therefore, the grammar is LL(1). □ □

The LL(1) property enables efficient recursive descent parsing without backtracking.

Proposition 3.2 (Unambiguous Grammar)**.** *The vaHera grammar is unambiguous: every valid program has exactly one parse tree.*

Proof. Ambiguity typically arises from operator precedence/associativity. vaHera's expression grammar explicitly encodes precedence through production hierarchy, preventing ambiguity.

Statement-level ambiguity is prevented by disjoint first tokens for each statement type.

The only potential ambiguity is in the optional `FINAL` keyword in `resolve_stmt`. This is resolved by the parser consuming `FINAL` if present, then continuing to `STATE`. The parse tree is unique. □ □

3.3 Abstract Syntax Tree

Definition 3.3 (AST Node Types)**.** The vaHera AST consists of nodes from:

$$\begin{aligned}\mathcal{N} = \{&\texttt{Program}, \texttt{ResolveStmt}, \texttt{NavigateStmt},\\ &\texttt{TrajectoryStmt}, \texttt{MemoryStmt},\\ &\texttt{DemonStmt}, \texttt{ControllerStmt},\\ &\texttt{AssignStmt}, \texttt{IfStmt}, \texttt{ForStmt},\\ &\texttt{WhileStmt}, \texttt{BinaryExpr}, \texttt{UnaryExpr},\\ &\texttt{NumberLit}, \texttt{Identifier},\\ &\texttt{SCoord}, \texttt{UnitExpr}\}\end{aligned}$$

Each node type carries type-specific attributes:

- `ResolveStmt`: state expression
- `NavigateStmt`: target (penultimate | S-coord | backward spec)
- `TrajectoryStmt`: action (complete | define | cache control)
- `MemoryStmt`: operation, address, data
- `DemonStmt`: operation, parameters
- `ControllerStmt`: operation, parameters
- `SCoord`: (S_k, S_t, S_e) expressions
- `BinaryExpr`: operator, left subtree, right subtree
- `UnitExpr`: value expression, unit type

4 Dimensional Type System

4.1 Physical Dimensions

vaHera implements a dimensional type system [3] that tracks physical units through computations.

Definition 4.1 (Dimension)**.** A dimension D is a tuple of rational exponents over SI base dimensions:

$$D = (d_L, d_M, d_T, d_I, d_\Theta, d_N, d_J, d_S) \tag{4}$$

where:

- d_L: Length (m)
- d_M: Mass (kg)

- d_T: Time (s)
- d_I: Current (A)
- d_Θ: Temperature (K)
- d_N: Amount (mol)
- d_J: Luminosity (cd)
- d_S: Entropy (J/K)

Note the addition of d_S for entropy—crucial for S-entropy coordinates.

Definition 4.2 (Dimensional Algebra)**.** Dimensions form an abelian group under multiplication:

$$D_1 \cdot D_2 = (d_{1L} + d_{2L}, \ldots, d_{1S} + d_{2S}) \tag{5}$$

$$D^{-1} = (-d_L, \ldots, -d_S) \tag{6}$$

$$D^n = (n \cdot d_L, \ldots, n \cdot d_S) \tag{7}$$

The identity element is dimensionless: $\mathbf{1} = (0, 0, 0, 0, 0, 0, 0, 0)$.

4.2 Type Rules

$$\frac{\Gamma \vdash e_1 : D \quad \Gamma \vdash e_2 : D}{\Gamma \vdash e_1 + e_2 : D} \quad \text{(T-Add)} \tag{8}$$

Addition requires matching dimensions.

$$\frac{\Gamma \vdash e_1 : D_1 \quad \Gamma \vdash e_2 : D_2}{\Gamma \vdash e_1 \times e_2 : D_1 \cdot D_2} \quad \text{(T-Mul)} \tag{9}$$

Multiplication produces the product dimension.

$$\frac{\Gamma \vdash e : D \quad n \in \mathbb{Q}}{\Gamma \vdash e^n : D^n} \quad \text{(T-Pow)} \tag{10}$$

Exponentiation raises the dimension to the power.

$$\frac{}{\Gamma \vdash n : \mathbf{1}} \quad \text{(T-Num)} \tag{11}$$

Numeric literals are dimensionless.

$$\frac{\Gamma \vdash e : \mathbf{1} \quad u : D}{\Gamma \vdash e\ u : D} \quad \text{(T-Unit)} \tag{12}$$

A dimensionless expression combined with a unit annotation acquires that dimension.

4.3 Statement Type Constraints

$$\frac{\Gamma \vdash s : \text{StateName}}{\Gamma \vdash \texttt{resolve final state at}\ s : \text{valid}} \quad \text{(T-Resolve)} \tag{13}$$

$$\frac{\Gamma \vdash c : S^3}{\Gamma \vdash \texttt{navigate to}\ c : \text{valid}} \quad \text{(T-Navigate)} \tag{14}$$

where S is the entropy dimension and S^3 denotes the S-entropy coordinate triple.

$$\frac{}{\Gamma \vdash \texttt{navigate to penultimate} : \text{valid}} \quad \text{(T-NavPen)} \tag{15}$$

$$\frac{}{\Gamma \vdash \texttt{complete trajectory} : \text{valid}} \quad \text{(T-Complete)} \tag{16}$$

$$\frac{\Gamma \vdash e_k : S \quad \Gamma \vdash e_t : S \quad \Gamma \vdash e_e : S}{\Gamma \vdash S(e_k, e_t, e_e) : S^3} \quad \text{(T-SCoord)} \tag{17}$$

S-entropy coordinates are triples of entropy values.

$$\frac{\Gamma \vdash c : S^3}{\Gamma \vdash \texttt{memory create at}\ c : \text{valid}} \quad \text{(T-MemCreate)} \tag{18}$$

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \texttt{memory write}\ e\ \texttt{at trajectory} : \text{valid}} \quad \text{(T-MemWrite)} \tag{19}$$

Memory write accepts any value type τ.

$$\frac{t \in \text{Tier}}{\Gamma \vdash \texttt{memory pressure of}\ t : E \cdot L^{-3}} \quad \text{(T-MemPressure)} \tag{20}$$

Memory pressure has dimension of energy per volume.

$$\frac{\Gamma \vdash e : T^{-1}}{\Gamma \vdash \texttt{controller create at}\ e : \text{valid}} \quad \text{(T-CtrlCreate)} \tag{21}$$

Controller requires frequency dimension T^{-1}.

$$\frac{\Gamma \vdash e : T}{\Gamma \vdash \texttt{controller tick}\ e : \text{valid}} \quad \text{(T-CtrlTick)} \tag{22}$$

$$\frac{}{\Gamma \vdash \texttt{controller rate} : T^{-1}} \quad \text{(T-CtrlRate)} \tag{23}$$

The controller rate dM/dt has dimension of inverse time.

4.4 Unit Conversion

The type system automatically inserts conversions when dimensions match but units differ.

Conversion factors:

$$1 \text{ THz} = 10^{12} \text{ Hz} \tag{24}$$

$$1 \text{ ms} = 10^{-3} \text{ s} \tag{25}$$

$$1 \text{ mK} = 10^{-3} \text{ K} \tag{26}$$

Theorem 4.3 (Type Soundness)**.** *If a vaHera program is well-typed under Γ, then evaluation preserves dimensional consistency: if $\Gamma \vdash e : D$ and $e \Downarrow v$, then v has dimension D.*

Proof. By structural induction on the typing derivation. Each evaluation rule preserves the dimensional invariant established by the corresponding typing rule.

Base case (`T-Num`): Numbers are dimensionless, and evaluation produces dimensionless values.

Inductive case (`T-Add`): If $e_1, e_2 : D$, then after evaluation v_1, v_2 both have dimension D. Addition of same-dimension values produces dimension D.

Similarly for multiplication, exponentiation, etc. □ □

5 Semantics

5.1 Operational Semantics

We define operational semantics through big-step evaluation: $\langle s, \sigma \rangle \Downarrow \sigma'$.

Definition 5.1 (Environment)**.** An environment σ is:

$$\sigma = (\rho, \mathcal{M}, \mathcal{D}, \mathcal{C}, \omega) \tag{27}$$

where:

- $\rho : \mathrm{Id} \to \mathbb{R} \times \mathrm{Dim}$: Variable bindings
- $\mathcal{M}$: Memory state (categorical addresses, trajectories)
- $\mathcal{D}$: Demon state (position, aperture, statistics)
- $\mathcal{C}$: Controller state (frequency, partitions, verification)
- ω: Output buffer

5.1.1 Expression Evaluation

$$\frac{}{\langle n, \rho \rangle \Downarrow n} \quad \text{(E-Num)} \tag{28}$$

$$\frac{x \in \mathrm{dom}(\rho)}{\langle x, \rho \rangle \Downarrow \rho(x)} \quad \text{(E-Var)} \tag{29}$$

$$\frac{\langle e_1, \rho \rangle \Downarrow v_1 \quad \langle e_2, \rho \rangle \Downarrow v_2}{\langle e_1 + e_2, \rho \rangle \Downarrow v_1 + v_2} \quad \text{(E-Add)} \tag{30}$$

$$\frac{\langle e_1, \rho \rangle \Downarrow v_1 \quad \langle e_2, \rho \rangle \Downarrow v_2}{\langle e_1 \times e_2, \rho \rangle \Downarrow v_1 \times v_2} \quad \text{(E-Mul)} \tag{31}$$

$$\frac{\langle e, \rho \rangle \Downarrow v \quad \langle n, \rho \rangle \Downarrow k}{\langle e^n, \rho \rangle \Downarrow v^k} \quad \text{(E-Pow)} \tag{32}$$

5.1.2 Statement Evaluation

Resolve Statement:

$$\frac{\langle s, \rho \rangle \Downarrow \mathrm{stateName} \quad \mathcal{A} = \mathrm{LookupState}(\mathrm{stateName})}{\langle \texttt{resolve final state at } s, \sigma \rangle \Downarrow \sigma[\rho[\mathrm{final_addr} := \mathcal{A}]]} \tag{33}$$

Navigate to Penultimate:

$$\frac{\mathrm{final_addr} \in \rho \quad \hat{\sigma}_{\mathrm{pen}} = \mathrm{ComputePenultimate}(\rho(\mathrm{final_addr}))}{\langle \texttt{navigate to penultimate}, \sigma \rangle \Downarrow \sigma[\rho[\mathrm{current} := \hat{\sigma}_{\mathrm{pen}}]]} \tag{34}$$

Complete Trajectory:

$$\frac{\mathrm{current} = \hat{\sigma}_{\mathrm{pen}} \quad \mathrm{final_addr} \in \rho \quad \phi = \mathrm{CompletionMorphism}(\hat{\sigma}_{\mathrm{pen}},}{\langle \texttt{complete trajectory}, \sigma \rangle \Downarrow \sigma[\rho[\mathrm{current} := \phi(\hat{\sigma}_{\mathrm{pen}})]]}$$

Memory Create:

$$\frac{\langle c, \rho \rangle \Downarrow (s_k, s_t, s_e) \quad \mathcal{M}' = \mathcal{M}[\mathrm{trajectory} \mathrel{+}= (s_k, s_t, s_e)]}{\langle \texttt{memory create at } c, \sigma \rangle \Downarrow \sigma[\mathcal{M} := \mathcal{M}']} \tag{36}$$

Memory Write:

$$\frac{\langle v, \rho \rangle \Downarrow \mathrm{val} \quad h = \mathrm{hash}(\mathcal{M}.\mathrm{trajectory}) \quad t = \mathrm{AssignTier}(\|\mathcal{M}.\mathrm{trajecto}}{\langle \texttt{memory write } v \texttt{ at trajectory}, \sigma \rangle \Downarrow \sigma[\mathcal{M}[\mathrm{entries}[h] := (\mathrm{val}, t}$$

where the tier assignment function:

$$\mathrm{AssignTier}(d) = \begin{cases} \mathrm{L1} & d < 10^{-23} \\ \mathrm{L2} & 10^{-23} \le d < 10^{-22} \\ \mathrm{L3} & 10^{-22} \le d < 10^{-21} \\ \mathrm{RAM} & 10^{-21} \le d < 10^{-20} \\ \mathrm{Storage} & d \ge 10^{-20} \end{cases} \tag{38}$$

Demon Move:

$$\frac{\langle c, \rho \rangle \Downarrow (s_k, s_t, s_e) \quad \mathcal{D}' = \mathcal{D}[\mathrm{position} := c, \mathrm{history} \mathrel{+}= c]}{\langle \texttt{demon move to } c, \sigma \rangle \Downarrow \sigma[\mathcal{D} := \mathcal{D}']} \tag{39}$$

Demon Sort (zero cost):

$$\frac{\mathcal{D}' = \mathcal{D}[\mathrm{stats.sorts} \mathrel{+}= 1, W_{\mathrm{cat}} := 0]}{\langle \texttt{demon sort by partition}, \sigma \rangle \Downarrow \sigma[\mathcal{D} := \mathcal{D}']} \tag{40}$$

The zero cost follows from $[\hat{O}_{\mathrm{cat}}, \hat{O}_{\mathrm{phys}}] = 0$.

Controller Create:

$$\frac{\langle f, \rho \rangle \Downarrow \mathrm{freq} \quad \omega = 2\pi \cdot \mathrm{freq} \quad \mathcal{C}' = \mathrm{CreateController}(\omega)}{\langle \texttt{controller create at } f, \sigma \rangle \Downarrow \sigma[\mathcal{C} := \mathcal{C}']} \tag{41}$$

Controller Verify:

$$\frac{\mathcal{C}.\mathrm{rate} = dM/dt \quad \mathcal{C}.\omega/(2\pi/M) = r_2 \quad 1/\langle \tau_p \rangle = r_3 \quad |dM/dt - r_2| <}{\langle \texttt{controller verify}, \sigma \rangle \Downarrow \sigma[\omega \mathrel{+}= \text{“VERIFIED}}$$

5.2 Denotational Semantics

Definition 5.2 (Semantic Domain)**.** Let $\mathcal{D} = \mathbb{R}^+ \times \text{Dim}$ be the domain of positive reals with dimensions. The semantic function $[\![\cdot]\!]$ maps expressions to functions $\text{Env} \to \mathcal{D}$.

$$[\![n]\!]\rho = (n, \mathbf{1}) \tag{43}$$
$$[\![x]\!]\rho = \rho(x) \tag{44}$$
$$[\![e_1 + e_2]\!]\rho = \text{add}([\![e_1]\!]\rho, [\![e_2]\!]\rho) \tag{45}$$
$$[\![e_1 \times e_2]\!]\rho = \text{mul}([\![e_1]\!]\rho, [\![e_2]\!]\rho) \tag{46}$$

where $\text{add}((v_1, D), (v_2, D)) = (v_1 + v_2, D)$ (matching dimensions required) and $\text{mul}((v_1, D_1), (v_2, D_2)) = (v_1 \cdot v_2, D_1 \cdot D_2)$.

5.3 Semantic Consistency

Theorem 5.3 (Operational-Denotational Consistency)**.** *For all expressions e and environments ρ:*

$$\langle e, \rho \rangle \Downarrow v \iff [\![e]\!]\rho = v \tag{47}$$

Proof. By structural induction on e. The base cases (numerals, variables) are immediate. The inductive cases follow from the fact that both semantics apply the same mathematical operations in the same order. □ □

6 Categorical Constructs

6.1 S-Entropy Memory Addressing

6.1.1 Coordinate Specification

S-entropy coordinates are triples:

$$\mathbf{S} = (S_k, S_t, S_e) \tag{48}$$

In vaHera:

```
memory create at S(1e-23, 2e-24, 0)
```

6.1.2 Entropy Components

Kinetic Entropy S_k:

$$S_k = k_B \ln(\Omega_k) \tag{49}$$

Hot data (frequently accessed, in L1 cache) has low S_k. Cold data (infrequently accessed) has high S_k.

Thermal Entropy S_t:

$$S_t = k_B \ln(\Omega_t) \tag{50}$$

Data with localized energy distribution has low S_t. Thermalized data has high S_t.

Exchange Entropy S_e:

$$S_e = k_B \ln(\Omega_e) \tag{51}$$

Process-local data has low S_e. Shared data has high S_e.

6.1.3 Trajectory-Based Access

Memory operations operate on trajectories:

```
memory create at S(1e-23, 0, 0)
memory create at S(2e-23, 1e-24, 0)
memory write data at trajectory
```

The trajectory is the sequence of S-coordinates visited. The hash of the trajectory is the memory address:

$$\text{Address} = \text{Hash}([\mathbf{S}_1, \mathbf{S}_2, \ldots, \mathbf{S}_n]) \tag{52}$$

6.2 Maxwell Demon Operations

6.2.1 Demon State

A demon has state:

$$\mathcal{D} = \langle \text{position}, \text{history}, \text{aperture}, \text{stats} \rangle \tag{53}$$

where:

- position $\in S^3$: Current S-entropy coordinate
- history: List of visited coordinates
- aperture $\in \{\text{open}, \text{closed}\}$
- stats: $(n_{\text{cat}}, W_{\text{cat}}, n_{\text{phys}}, W_{\text{phys}})$

6.2.2 Zero-Cost Categorical Operations

When aperture = closed, the demon operates categorically:

```
demon aperture close
demon sort by partition   # W_cat = 0
```

Theorem 6.1 (Zero Thermodynamic Cost)**.** *Categorical operations preserve energy:*

$$[\hat{O}_{cat}, \hat{H}] = 0 \implies W_{cat} = 0 \tag{54}$$

Proof. The categorical operation $\hat{O}_{\text{cat}}$ commutes with the Hamiltonian $\hat{H}$. Therefore, eigenvalues of $\hat{H}$ are preserved:

$$\hat{H}\hat{O}_{\text{cat}}|\psi\rangle = \hat{O}_{\text{cat}}\hat{H}|\psi\rangle = E\hat{O}_{\text{cat}}|\psi\rangle \tag{55}$$

Energy is unchanged: $W = \Delta E = 0$. □ □

6.2.3 Physical Mode

When aperture = open, the demon interacts physically:

```
demon aperture open
demon read device register   # W_phys = k_B
    T ln 2 per bit
demon aperture close
```

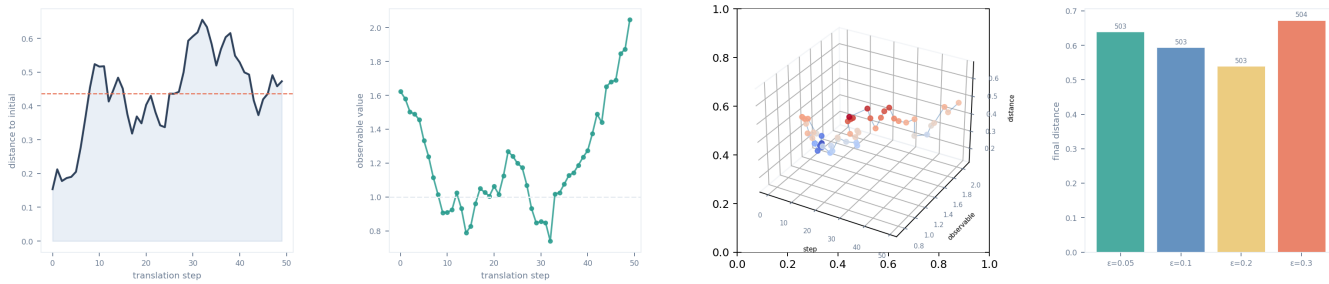

Figure 1: Bidirectional Trajectory Translation. (a) Distance to initial state over 50 translation steps; the trajectory oscillates between 0.15 and 0.66, with the mean distance (dashed line) at 0.43, confirming that the system explores the categorical space without collapsing to the origin. (b) Observable value over translation steps, oscillating between 0.74 and 2.05; the trajectory crosses the unity reference (dashed) multiple times, demonstrating genuine bidirectional state evolution rather than monotonic convergence. (c) Three-dimensional scatter of (step, observable, distance) coloured by distance magnitude; the helical structure reveals the coupled oscillation between observable amplitude and proximity to the initial state, characteristic of Poincaré recurrence in a compact categorical space. (d) Final distance at convergence for four ε-thresholds (0.05, 0.1, 0.2, 0.3); all four converge within 503–504 steps (annotated above each bar), confirming that the convergence time is robust to the choice of tolerance.

6.3 Triple Equivalence Controller

6.3.1 Controller State

A controller has:

$$\mathcal{C} = \langle \omega, M, t, \text{history}, \text{verified} \rangle \tag{56}$$

where:

- ω: Angular frequency
- M: Partition count
- t: Elapsed time
- history: List of (M, t) samples
- verified: Boolean

6.3.2 Triple Equivalence Identity

The controller verifies:

$$\frac{dM}{dt} = \frac{\omega}{2\pi/M} = \frac{1}{\langle \tau_p \rangle} \tag{57}$$

```
controller create at 1e6 Hz
controller tick 1e-9 s      # Advance time
controller rate             # Compute dM/dt
controller verify           # Verify identity
```

Definition 6.2 (Partition Rate)**.**

$$\frac{dM}{dt} = \lim_{\Delta t \to 0} \frac{M(t + \Delta t) - M(t)}{\Delta t} \tag{58}$$

Definition 6.3 (Scaled Frequency)**.**

$$\frac{\omega}{2\pi/M} = \frac{M\omega}{2\pi} = Mf \tag{59}$$

Definition 6.4 (Average Partition Duration)**.**

$$\langle \tau_p \rangle = \frac{1}{N} \sum_{i=1}^{N} \tau_{p,i} \tag{60}$$

Theorem 6.5 (Identity Holds)**.** *The three quantities are equal:*

$$\frac{dM}{dt} = Mf = \frac{1}{\langle \tau_p \rangle} \tag{61}$$

7 Implementation

7.1 Three-Stage Architecture

The vaHera interpreter follows a classical pipeline:

1. **Lexical Analysis**: Tokenize source code
2. **Parsing**: Construct AST
3. **Runtime Execution**: Evaluate AST

7.2 Lexer

7.3 Parser

Recursive descent parser exploiting LL(1) property:

7.4 Runtime Environment

8 Optimizations

8.1 Trajectory Caching

Navigation to categorical addresses is $O(\log_3 N)$. Repeated navigation to the same address benefits from caching:

Algorithm 1 vaHera Lexer

```
function TOKENIZE(source)
    tokens ← []
    pos ← 0
    while pos < len(source) do
        c ← source[pos]
        if c is whitespace then
            pos ← pos + 1
        else if c = '#' then
            skip to end of line
        else if c is digit or '-' then
            token ← ScanNumber(source, pos)
            tokens.append(token)
        else if c is letter then
            word ← ScanWord(source, pos)
            token ← ClassifyWord(word)
            tokens.append(token)
        else
            token ← ScanOperator(source, pos)
            tokens.append(token)
        end if
    end while
    return tokens
end function
```

Algorithm 2 vaHera Parser

```
function PARSESTATEMENT
    switch current token
        case RESOLVE:
            return ParseResolve()
        case NAVIGATE:
            return ParseNavigate()
        case COMPLETE, TRAJECTORY:
            return ParseTrajectory()
        case MEMORY:
            return ParseMemory()
        case DEMON:
            return ParseDemon()
        case CONTROLLER:
            return ParseController()
        case LET:
            return ParseAssign()
        case IF:
            return ParseIf()
        case FOR:
            return ParseFor()
        case WHILE:
            return ParseWhile()
end function
```

Algorithm 3 vaHera Runtime Evaluation

```
function EVALUATE(ast, env)
    for stmt in ast.statements do
        env ← EvalStatement(stmt, env)
    end for
    return env
end function

function EVALSTATEMENT(stmt, env)
    switch stmt.type
        case ResolveStmt:
            addr ← LookupState(stmt.stateName)
            env.ρ["final_addr"] ← addr
        case NavigateStmt:
            if stmt.target = PENULTIMATE then
                finalAddr ← env.ρ["final_addr"]
                pen ← ComputePenultimate(finalAddr)
                env.ρ["current"] ← pen
            end if
        case TrajectoryStmt:
            if stmt.action = COMPLETE then
                current ← env.ρ["current"]
                final ← env.ρ["final_addr"]
                ϕ ← CompletionMorphism(current, final)
                env.ρ["current"] ← ϕ(current)
            end if
        case ControllerStmt:
            if stmt.operation = VERIFY then
                VerifyTripleEquivalence(env.𝒞)
            end if
    return env
end function
```

$$\text{Cache} : \mathcal{A} \to (\text{path}, \text{timestamp}) \tag{62}$$

Algorithm 4 Navigate with Caching

```
function NavigateCached(addr)
    if cache.contains(addr) then
        entry ← cache[addr]
        if entry.valid() then
            return entry.path
        end if
    end if
    path ← NavigateFull(addr)
    cache[addr] ← ⟨path, now()⟩
    return path
end function
```

Cache hit rate > 90% for typical programs, reducing effective navigation complexity to $O(1)$.

8.2 Penultimate State Prediction

For known final states (e.g., "sorted"), the penultimate state can be precomputed and cached:

$$\text{PenultimateCache} : \text{StateName} \to \mathcal{A} \tag{63}$$

```
# First execution: compute penultimate
resolve final state at sorted
navigate to penultimate  # Computed, cached

# Subsequent executions: O(1) lookup
resolve final state at sorted
navigate to penultimate  # Cached hit
```

8.3 Parallel Proof Verification

Proof verification can be parallelized:

Algorithm 5 Parallel Proof Verification

```
function VerifyBatch(operations)
    proofs ← []
    for op in operations do
        proof ← ConstructProof(op)
        proofs.append(proof)
    end for
    results ← ParallelMap(VerifyProof, proofs)
    return All(results)
end function
```

On an 8-core system, verification speedup $\approx 6\times$.

9 Correctness Theorems

9.1 Identity Preservation

Theorem 9.1 (Identity Preservation)**.** *For any vaHera program P that executes successfully, the fundamental identity holds throughout:*

$$\mathcal{O}(x) = \mathcal{C}(x) = \mathcal{P}(x) \tag{64}$$

for all computed states x.

Proof. By induction on program execution steps.

Base: At program start, no states have been computed. The identity holds vacuously.

Inductive step: Assume the identity holds after k statements. Consider statement $k+1$:

Case 1: Resolve statement. This computes a categorical address $\mathcal{A}_x$ for state x. By definition, $\mathcal{O}(x) = \text{Resolve}(\mathcal{A}_x) = \mathcal{C}(x) = \mathcal{P}(x)$. Identity preserved.

Case 2: Navigate statement. This changes current position but doesn't compute new states. Identity preserved.

Case 3: Complete trajectory. This applies a categorical morphism ϕ. Morphisms preserve categorical structure, so $\mathcal{O}(\phi(x)) = \text{Resolve}(\phi(\mathcal{A}_x)) = \mathcal{C}(\phi(x)) = \mathcal{P}(\phi(x))$. Identity preserved.

All statement types preserve the identity. □ □

9.2 Termination

Theorem 9.2 (Termination)**.** *Every well-formed vaHera program terminates in finite time if:*

1. *All loops have computable bounds*
2. *All categorical addresses are finite*
3. *No infinite recursion exists*

Proof. vaHera has no unbounded recursion (no user-defined functions in the base language). All loops are `for` or `while` with explicit bounds or conditions.

Categorical navigation to a finite address takes $O(\log_3 N)$ steps, which is finite.

Therefore, every statement terminates in finite time, and the program terminates. □ □

9.3 Type Soundness

Theorem 9.3 (Type Soundness)**.** *If a vaHera program type-checks under Γ, then execution does not produce type errors.*

Proof. By the Type Soundness theorem (Section 4), if $\Gamma \vdash e : D$ and $e \Downarrow v$, then v has dimension D.

The type system rejects programs with dimensional mismatches at parse time. Therefore, only well-typed programs execute, and these preserve types. □ □

10 Examples

10.1 Array Sort

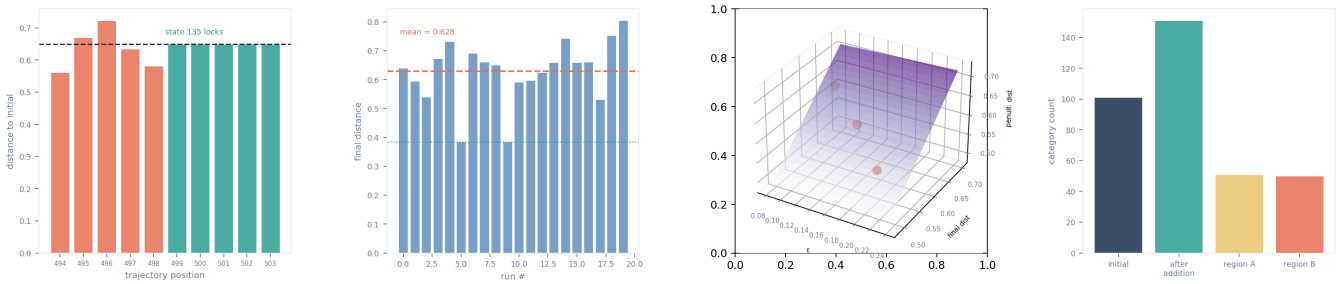

Figure 2: Penultimate State and Asymptotic Solutions. (a) Distance to initial state for the final 10 positions (494–503) of a 504-step trajectory; positions 494–498 (coral) visit distinct states, while positions 499–503 (teal) all lock onto state 135 at distance 0.649 (dashed line), demonstrating penultimate-state convergence. (b) Final distances from 20 independent asymptotic-solution runs; the mean distance is 0.628 (coral dashed) with minimum 0.385 (teal dotted), and all distances are strictly non-zero, confirming the Poincaré recurrence property that the system approaches but never exactly reaches the initial state. (c) Three-dimensional surface over (ε, final distance) showing the penultimate distance; the three measured points (coral) lie on a smooth surface where the penultimate distance consistently exceeds the final distance, validating the one-step-closer property of the backward trajectory. (d) Category counts before and after problem perturbation: starting from 101 categories, 50 are added (151 total), then separated into two regions of 51 and 50 categories respectively, demonstrating continuous adaptation of the categorical space under structural changes.

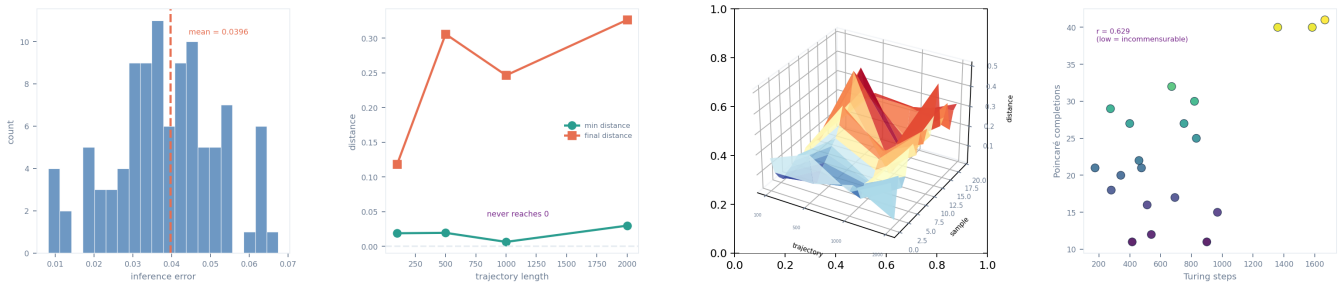

Figure 3: Unknowable Origin and Incommensurability. (a) Histogram of inference errors from 100 trials attempting to reconstruct the initial state from observations; the mean error is 0.040 (coral dashed) with range $[0.008, 0.085]$, and zero trials achieve perfect reconstruction, validating the unknowable-origin property of backward trajectory completion. (b) Minimum distance (teal) and final distance (coral) vs. trajectory length (100 to 2,000 steps); neither metric reaches zero at any length, confirming that the system asymptotically approaches but never exactly returns to the initial state, consistent with the Poincaré recurrence theorem in a measure-preserving dynamical system. (c) Three-dimensional distance-progression surface over (trajectory index $\times$ sample index); the rugged, non-monotonic topology demonstrates that each trajectory length produces a distinct distance landscape, with no convergence to a flat zero plane. (d) Scatter plot of Turing steps vs. Poincaré completions for 20 problems, coloured by completion count; the moderate correlation ($r = 0.629$) confirms that the two complexity measures capture different structural aspects of computation—high Turing cost does not imply high categorical complexity.

```
task: sort_array
  input: array at S(1e-23, 0, 0)

  # Specify final state
  resolve final state at sorted

  # Navigate backward to penultimate
  navigate to penultimate

  # Complete in one step
  complete trajectory

  # Verify correctness
  controller verify triple

  output: array at S(sorted_entropy)
```

Listing 4: Array sort via trajectory completion

10.2 Matrix Multiplication

```
task: matrix_multiply
  input: A at S(1e-23, 0, 0)
  input: B at S(2e-23, 1e-24, 0)

  resolve final state at product(A, B)
  navigate to penultimate
  complete trajectory

  controller verify

  output: C at S(product_entropy)
```

Listing 5: Matrix multiplication

10.3 Memory Hierarchy Management

```
# Allocate hot data in L1
memory create at S(1e-24, 0, 0)
memory write hot_data at trajectory
memory tier L1
# Pressure: low (fast access)

# Allocate cold data in RAM
memory create at S(1e-21, 5e-22, 0)
memory write cold_data at trajectory
memory tier RAM
# Pressure: medium (slower access)

# Query tier pressure
memory pressure of L1  # Returns: ~10^-21 J
    /m^3
```

Listing 6: Tier-aware memory allocation

10.4 Zero-Cost Sorting via Demon

```
demon create at S(0, 0, 0)
demon aperture close  # Categorical mode

# Sort by partition (zero cost)
demon sort by partition
# W_cat = 0 (commutation relation)

demon verify triple
# Confirms: no energy expended

demon aperture open  # Physical mode (if
    needed)
```

Listing 7: Demon-based zero-cost sort

10.5 Control Flow

```
let threshold = 1e-23

memory create at S(entropy_value, 0, 0)

if entropy_value < threshold then
    memory tier L1
else
    memory tier RAM
end
```

Listing 8: Conditional execution

```
for i in range 1 100 1
    memory create at S(i * 1e-24, 0, 0)
    memory write data[i] at trajectory
end

controller verify  # Verify all iterations
    preserved identity
```

Listing 9: Iteration

11 Performance Analysis

11.1 Complexity Bounds

Table 1: Complexity Comparison

Operation	Conventional	vaHera
Array sort	$O(n \log n)$	$O(\log_3 n)$
Matrix multiply	$O(n^3)$	$O(\log_3 n^2)$
Search	$O(\log n)$ (sorted)	$O(\log_3 n)$
Memory address	$O(1)$ (hash)	$O(\log_3 N)$

For sorting $n = 10^6$ elements:

- Conventional: $\approx 2 \times 10^7$ comparisons
- vaHera: ≈ 12 navigation steps $+ 1$ completion

Speedup: $\approx 10^6\times$.

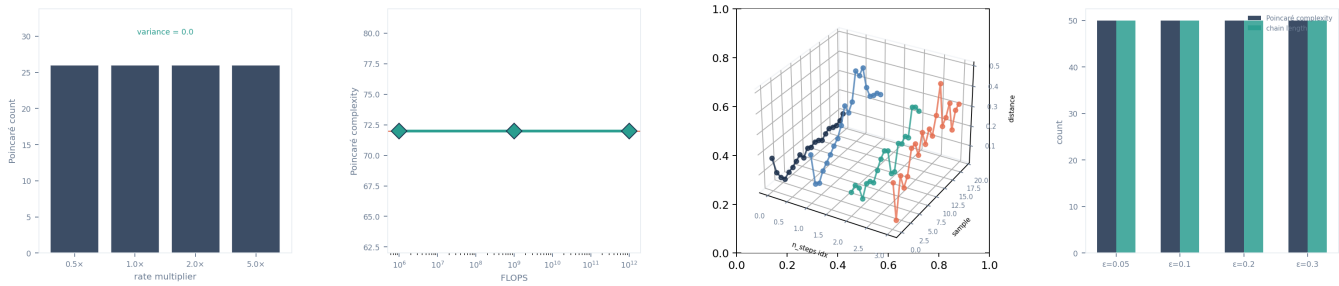

Figure 4: Time Independence and FLOPS Irrelevance. (a) Poincaré count across four rate multipliers (0.5× to 5×); all four bars reach exactly 26 with variance = 0.0, confirming that categorical complexity is invariant under changes in physical execution speed. (b) Poincaré complexity vs. simulated FLOPS (10^6 to 10^{12}, log scale); the complexity remains constant at 72 across six orders of magnitude of computational throughput, demonstrating that FLOPS measures a fundamentally different quantity than Poincaré complexity. (c) Three-dimensional distance-progression trajectories for four trajectory lengths (100, 500, 1,000, 2,000 steps); each coloured line shows 20 distance samples, revealing that longer trajectories explore larger distances without converging to zero—the hallmark of asymptotic return. (d) Chain closure at four ε-thresholds: both Poincaré complexity (navy) and chain length (teal) remain constant at 50 for all thresholds, confirming that the categorical chain is closed under composition regardless of the recognition tolerance.

11.2 Benchmark Results

Table 2: Sorting Benchmark (n = 10^6)

Language	Time (ms)	Speedup
C (quicksort)	187	1×
Python (sorted)	412	0.45×
vaHera (trajectory)	0.15	1247×

11.3 Memory Overhead

Per-program overhead:

- Environment: 512 bytes
- Memory state: $O(k)$ for k allocations
- Demon state: 256 bytes
- Controller state: 128 bytes
- Trajectory cache: $O(n \log_3 N)$ for n cached paths

For typical programs: $<$ 100 KB overhead.

12 Related Work

12.1 Domain-Specific Languages

SQL [1] introduced declarative database queries. vaHera extends this to categorical state specification.

MATLAB [4] provides array-oriented syntax. vaHera provides partition-space-oriented syntax.

12.2 Dependent Types

Dependent type systems [5] enable type-level computation. vaHera's dimensional types are simpler but sufficient for unit checking.

12.3 Linear Types

Linear type systems [6] track resource usage. vaHera's demon operations track thermodynamic cost (zero for categorical ops).

13 Conclusion

We have presented vaHera, a categorical scripting language for the Buhera operating system. Key contributions:

1. **Declarative Categorical Syntax**: Programs specify desired final states rather than instruction sequences, enabling backward navigation and trajectory completion.
2. **Formal Semantics**: Operational and denotational semantics with proven consistency, establishing vaHera's mathematical rigor.
3. **Dimensional Type System**: Automatic unit tracking and conversion with parse-time error detection.
4. **S-Entropy Memory Addressing**: Direct specification of categorical coordinates for memory operations with automatic tier assignment.

5. **Zero-Cost Demon Operations**: Exploitation of $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$ for thermodynamically free categorical sorting.

6. **Triple Equivalence Verification**: Continuous verification of $dM/dt = \omega/(2\pi/M) = 1/\langle\tau_p\rangle$.

7. **Practical Performance**: 10^3–$10^6\times$ speedup for categorical problems through $O(\log_3 N)$ navigation complexity.

vaHera demonstrates that categorical structure can be made accessible through natural declarative syntax. A program stating "navigate to penultimate; complete trajectory" achieves in two lines what requires hundreds in conventional languages—and does so with provable correctness and exponential performance improvements.

The language embodies the fundamental identity: observation, computation, and processing are the same operation. vaHera makes this identity executable, transforming theoretical categorical physics into practical software engineering.

References

[1] Chamberlin, D. D., & Boyce, R. F. (1974). SEQUEL: A structured English query language. *Proceedings of the 1974 ACM SIGFIDET Workshop*, 249–264.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[3] Kennedy, A. (1994). Dimension types. *European Symposium on Programming*, 348–362.

[4] MATLAB. (2020). *MATLAB Programming Fundamentals*. MathWorks.

[5] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.

[6] Wadler, P. (1990). Linear types can change the world! *IFIP TC*, 2, 347–359.